Day 23, 12/11/26

234. Palindrome Linked List

Approach: Find Middle + Reverse Second Half.
uses slow and fast pointers and reverses the second half of the linked list.

1 → 2 → 2 → 1
First half: 1 → 2
second half: 2 → 1
Reverse second half: 1 → 2
Compare
1 == 1 , 2 == 2 → Palindrome

Code:-

```
Class Solution {
  public boolean isPalindrome(ListNode head){
    if(head == null || head.next == null){
      return true;
    }
    while.
    ListNode slow = head;
    ListNode fast = head;

    while(fast != null && fast.next != null){
      slow = slow.next;
      fast = fast.next.next;
    }
    ListNode secondHalf = reverse(slow);
    ListNode firstHalf = head;
```

```
        while (secondHalf != null)
            if (firstHalf.val != secondHalf.val) {
                return false;
            }
            firstHalf = firstHalf.next;
            secondHalf = secondHalf.next;
        }

        return true;
    }

    private ListNode reverse(ListNode head) {
        ListNode prev = null;

        while (head != null) {
            ListNode next = head.next;
            head.next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }
}
```

Time Complexity : O(n)
Space Complexity : O(1)

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 93 / 93 testcases passed

nisha_prajapati submitted at Nov 12, 2026 10:17

Analysis | Solution

Save 41%

Big Diwali Dhamaka

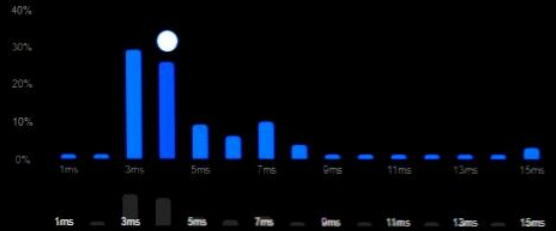
Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!

Runtime

4 ms | Beats 70.28%

Memory

94.02 MB | Beats 76.25%



Code | Java

```java
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
```

Code

Java | Auto

```java
class Solution {
    public boolean isPalindrome(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode secondHalf = reverse(slow);
        ListNode firstHalf = head;

        while (secondHalf != null) {
            if (firstHalf.val != secondHalf.val) {
                return false;
            }
            firstHalf = firstHalf.next;
            secondHalf = secondHalf.next;
        }
        return true;
    }
    private ListNode reverse(ListNode head) {
        ListNode prev = null;
        ListNode current = head;
        while (current != null) {
            ListNode next = current.next;
            current.next = prev;
            prev = current;
            current = next;
        }
        return prev;
    }
```

Saved

Ln 21, Col 9

Testcase | Test Result